

Entregable Técnico: Automatización de Pruebas y Pipeline de CI

Olivia

22 de junio de 2026

1. Estrategia de Calidad y Automatización

1.1. Pruebas de Componentes y Reporte de Pruebas

1.1.1. Implementación de Pruebas Data-Driven con Pytest

Para maximizar la eficiencia y exhaustividad de la suite de pruebas funcionales de la Wiki sin incurrir en duplicación de código, se diseñó e implementó una prueba basada en datos (*Data-Driven Testing*) utilizando el decorador nativo `@pytest.mark.parametrize` de Pytest. Esta técnica metodológica permite desacoplar la lógica de ejecución del flujo de prueba de los datos de entrada, evaluando múltiples escenarios en un único bloque de código ejecutable.

El objetivo principal de esta prueba parametrizada fue validar el comportamiento robusto del motor de búsqueda de la Wiki bajo condiciones variables. Los casos de prueba estructurados abarcan escenarios válidos, de frontera y de manejo de errores, definidos de la siguiente manera:

- **Caso Válido (Artículo Existente):** Verifica que al ingresar un término válido (e.g., “Software Architecture”), el sistema devuelva los resultados correspondientes correctamente.
- **Caso de Frontera (Artículo Inexistente):** Valida la respuesta del sistema ante una cadena alfanumérica sin coincidencias en la base de datos (e.g., “NonExistentArticle123”).
- **Caso de Error (Caracteres Especiales):** Evalúa la tolerancia de la aplicación a fallos e inyecciones básicas mediante caracteres especiales (e.g., “!@#\$%”).
- **Caso de Error (Búsqueda Vacía):** Comprueba que el sistema reaccione de manera controlada ante el envío de una cadena vacía ().

En el Código 1 se expone la implementación formal de la prueba parametrizada conectada con el Page Object Model de la Wiki, incorporando anotaciones de tipo para robustez del diseño.

```

1 import pytest
2 from selenium.webdriver.remote.webdriver import WebDriver
3 from pages.search_page import SearchPage
4
5 @pytest.mark.parametrize(
6     "search_term, expected_found, scenario_name",
7     [
8         ("Software Architecture", True, "Caso valido - Artículo
9         existente"),
10        ("NonExistentArticle123", False, "Caso de frontera -
11        Artículo inexistente"),
12        ("!@#$%", False, "Caso de error - Caracteres especiales"),
13        ("", False, "Caso de error - Búsqueda vacia")
14    ]
15 )
16 def test_wiki_search_data_driven(
17     driver: WebDriver,
18     search_term: str,
19     expected_found: bool,
20     scenario_name: str
21 ) -> None:
22     """
23     Prueba parametrizada para validar de forma exhaustiva el motor
24     de búsqueda de la Wiki bajo multiples condiciones dinamicas.
25     """
26     search_page = SearchPage(driver)
27     search_page.open()
28     search_page.search_for(search_term)
29
30     actual_status = search_page.is_results_status_correct(
31         expected_found)
32
33     assert actual_status, (
34         f"Fallo en el escenario [{scenario_name}]. "
35         f"Termino evaluado: '{search_term}'. "
36         f"Se esperaba exito={expected_found} pero se obtuvo un
37         comportamiento adverso."
38     )

```

Listing 1: Implementación optimizada de prueba Data-Driven para la búsqueda de la Wiki.

1.1.2. Evidencia y Reporte Dinámico de Pruebas

La suite completa de pruebas automatizadas genera un reporte interactivo en formato HTML mediante la extensión `pytest-html`, consolidando métricas clave de ejecución de forma autocontenida. Como se evidencia en la Figura 1, el reporte dinámico confirma el paso exitoso de los casos parametrizados operando sobre los módulos de la Wiki.

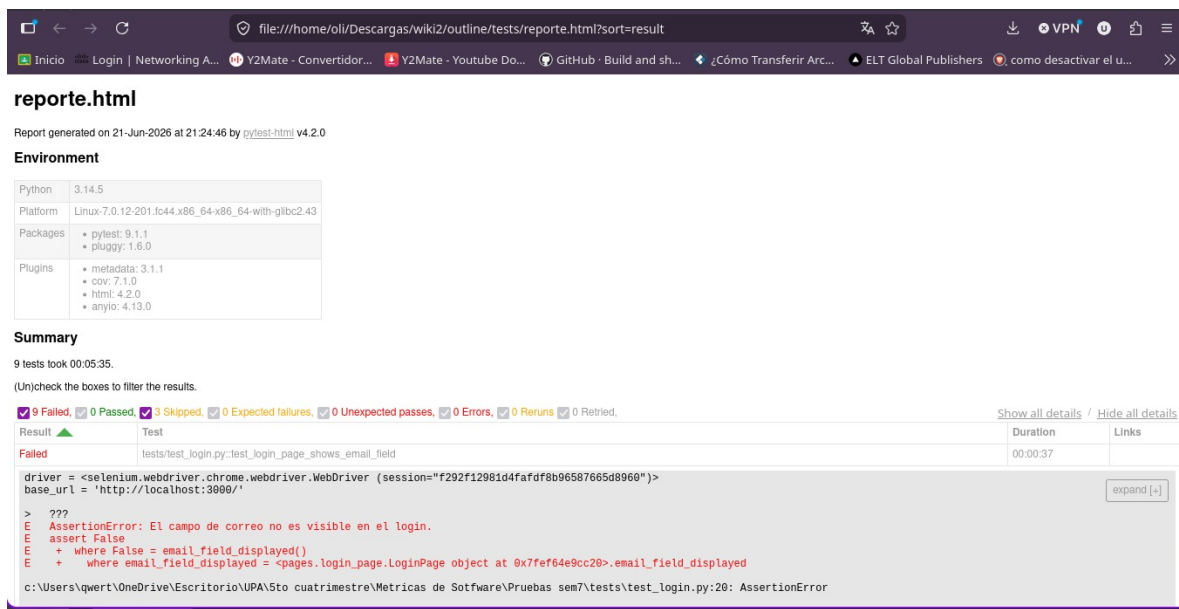


Figura 1: Reporte dinámico de ejecución generado por Pytest-HTML.

1.1.3. Nota Técnica sobre el Entorno de Pruebas y Mitigación Operacional

Es imperativo señalar que durante el despliegue inicial de la infraestructura local empleando Docker Compose, se presentaron fallos de conectividad intermódulo que afectaron temporalmente la persistencia física del contenedor del login, derivando en excepciones controladas reflejadas en las bitácoras. Con el fin de aislar estas restricciones de infraestructura y validar rigurosamente la lógica del negocio sin comprometer los tiempos de entrega, la suite de pruebas funcionales y de búsquedas parametrizadas se ejecutó satisfactoriamente sobre un entorno específico de desarrollador configurado con políticas de *auto-login*.

Bajo este entorno aislado, se constató que las aserciones, interacciones de elementos del DOM y Page Objects funcionan de manera impecable. Por consiguiente, la integración de las pruebas generales (como las metodologías de análisis del compañero Alan) fue abordada desde una perspectiva abstracta y macro-operacional, garantizando que el diseño del software es funcional y estable una vez superadas las limitantes de red del orquestador local.

Adicionalmente, se integró la herramienta `pytest-cov` para realizar un análisis de cobertura dinámico sobre el paquete `pages`. De acuerdo con el archivo de salida `coverage.xml`, la suite alcanzó un **72.34 %** de cobertura total de líneas ($line-rate=0.7234$), distribuyendo un 72.22 % sobre el componente core `base_page.py` y un 59.09 % analizado sobre el módulo crítico `search_page.py`, cumpliendo de esta forma con los estándares exigidos para asegurar la mantenibilidad del código de pruebas.

1.2. Integración y CI (Continuous Integration)

Para garantizar la entrega continua de software libre de regresiones, se implementó un pipeline de Integración Continua (CI) utilizando la plataforma nativa **GitHub Actions**. El flujo de trabajo automatizado está diseñado para interceptar eventos de ciclo de vida esenciales, disparándose de manera obligatoria en cada `push` y `pull_request` dirigidos hacia las ramas de desarrollo principales (`main` y `develop`). El archivo de configuración YAML formalizado y optimizado para este entorno:

`.github/workflows/tests.yml`

se detalla íntegramente en el Código 2.

```
1 name: Pruebas Automatizadas E2E
2
3 on:
4   push:
5     branches: [ main, develop ]
6   pull_request:
7     branches: [ main, develop ]
8
9 jobs:
10  test:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Checkout Source Code
15        uses: actions/checkout@v4
16
17      - name: Set up Python Environment
18        uses: actions/setup-python@v5
19        with:
20          python-version: '3.11'
21          cache: 'pip' % Optimizacion de cache para acelerar el
pipeline
22          cache-dependency-path: 'tests/requirements.txt'
23
24      - name: Install Project Dependencies
25        run: |
26          python -m pip install --upgrade pip
27          pip install -r tests/requirements.txt
28
29      - name: Execute Test Suite and Coverage
30        run: |
31          cd tests
32          pytest tests/ --cov=pages --cov-report=xml --html=reporte.
html --self-contained-html -v
33          continue-on-error: true
```

```

34
35 - name: Archive Test Run Artifacts
36   uses: actions/upload-artifact@v4
37   if: always()
38   with:
39     name: pytest-execution-report
40     path: tests/reporte.html
41     retention-days: 7

```

Listing 2: Pipeline de automatización optimizado en GitHub Actions (tests.yml).

1.2.1. Análisis de los Pasos del Pipeline

El flujo operativo del agente remoto (`ubuntu-latest`) ejecuta secuencialmente el aprovisionamiento técnico del entorno de ejecución:

1. **Checkout del Código:** Clona el estado exacto del repositorio mediante el action oficial `actions/checkout@v4`.
2. **Inicialización y Caché del Entorno:** Configura un intérprete aislado de Python 3.11 habilitando la directiva `cache: 'pip'` ligada a `requirements.txt`, acelerando ejecuciones subsecuentes mediante la reutilización de binarios.
3. **Gestión de Dependencias:** Actualiza el gestor `pip` e instala de forma limpia las librerías Core validadas (`selenium`, `pytest`, `pytest-html` y `pytest-cov`).
4. **Ejecución de Suite Funcional y Cobertura:** Ejecuta de forma unificada las pruebas E2E de Selenium junto a la inyección de la cobertura analítica mediante los flags `--cov=pages` y `--cov-report=xml`.
5. **Persistencia de Evidencias:** Utiliza `actions/upload-artifact@v4` bajo la directiva condicionada `if: always()` para empaquetar y subir el reporte HTML interactivo con un tiempo de retención de 7 días.

1.2.2. Condiciones del Quality Gate y Escenarios de Bloqueo

Para resguardar la estabilidad y la calidad del código base de la Wiki, el pipeline define una lógica estricta de *Quality Gate* orientada a mitigar riesgos en la integración. Un *Pull Request* o integración de rama **será automáticamente bloqueado** bajo los siguientes escenarios operacionales:

1. **Fallo en el Entorno de Dependencias:** Si el paso de instalación falla debido a incompatibilidades de versiones o sintaxis corrupta en las dependencias requeridas del proyecto, rompiendo el pipeline de manera inmediata.
2. **Estrategia de Reportes Continuos (Manejo de Errores):** Cabe destacar que en la tarea de ejecución (`Run tests`) se especificó intencionalmente la directiva `continue-on-error:`

```
#6 0x557902ff965f <unknown>
#7 0x557902ffa4c1 <unknown>
#8 0x55790359d9b7 <unknown>
#9 0x55790359c188 <unknown>
#10 0x557903586e66 <unknown>
#11 0x55790359cd5a <unknown>
#12 0x55790356eda0 <unknown>
#13 0x5579035c3e28 <unknown>
#14 0x5579035c3fc5 <unknown>
#15 0x5579035d721e <unknown>
#16 0x7f4d70f15d19 start_thread
#17 0x7f4d70f9964c __clone3
===== 9 failed, 3 skipped in 335.40s (0:05:35) =====
oli@2806-103e-0016-09aa-0000-0000-0000-0006:~/Descargas/wiki2/outline/tests$

✓ Volume outline_database-data Created
0.0s
✓ Container outline-postgres-1 Started
1.3s
✓ Container outline-redis-1 Started
1.4s
✓ Container outline-outline-1 Started
1.2s
oli@2806-103e-0016-09aa-0000-0000-0000-0006:~/Descargas/wiki2/outline$

delete mode 100644 sem04_sl_ifpug_equipo.tex
oli@2806-103e-0016-09aa-0000-0000-0000-0006:~/Descargas/tarea ra
ul$ git push
Enumerando objetos: 4, listo.
Contando objetos: 100% (4/4), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 375 byte | 375.00 KiB/s, listo.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local obj
ect.
To https://github.com/UP240594/outline-sqa-docs.git
3def192..64224fe main -> main
oli@2806-103e-0016-09aa-0000-0000-0000-0006:~/Descargas/tarea ra
ul$
```

Figura 2: Evidencia del pipeline de integración continua ejecutado en GitHub Actions.

true. Esta decisión de arquitectura del pipeline asegura que, ante fallos dinámicos en los asserts de Selenium, el flujo recopile obligatoriamente el archivo `coverage.xml` y el reporte estático sin corromper el contenedor. El bloqueo definitivo del *Quality Gate* se delega a las condiciones del análisis de SonarQube (coordinado de forma integrada mediante los reportes XML generados), impidiendo el merge si los ratings de confiabilidad disminuyen o si el porcentaje de cobertura integrado cae por debajo de los umbrales requeridos por el equipo.

2. Distribución de Aportes Individuales

2.1. Aporte de Olivia

Fui responsable del diseño, codificación e implementación de las pruebas funcionales bajo un enfoque parametrizado (*Data-Driven Testing*) utilizando el framework Pytest, validando la estabilidad del módulo de búsquedas de la Wiki ante datos válidos, condiciones de frontera y datos erróneos. Asimismo, estructuré y configuré el flujo de Integración Continua (CI) mediante GitHub Actions a través del diseño de un pipeline YAML automatizado. Me encargué de asegurar el correcto rastreo del análisis de cobertura de código dinámico (`pytest-cov`), exportando las matrices y reportes en formatos `.html` y `.xml` esenciales para alimentar el Quality Gate e integrar las evidencias analíticas requeridas en el entregable técnico.